

Análise Orientada a Objetos

7ª parte

Jobson Massollar
jobson.silva@uva.br

- Inicialmente o Analista pode descrever as funcionalidades do sistema usando UCs:

Nome: Emprestar livro

Objetivo: Realizar o empréstimo de livro ao aluno

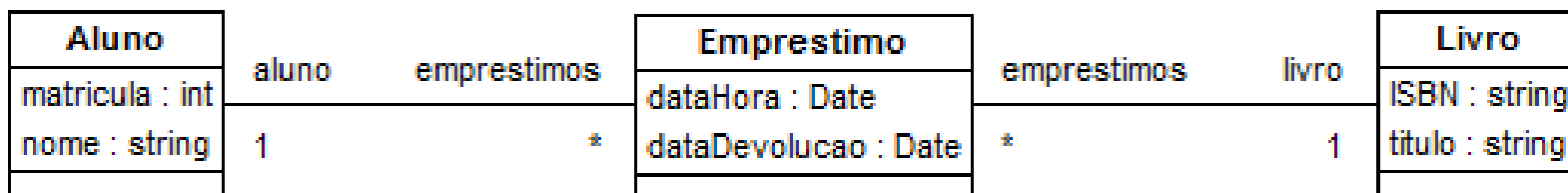
Ator: Bibliotecária

Trigger: Ator acessa a página de **empréstimo** de **livros**

Fluxo Principal:

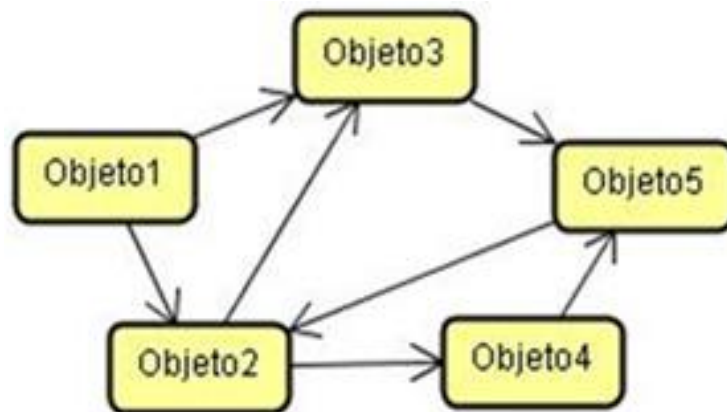
1. Sistema solicita a **matrícula** do **aluno**.
2. Ator informa a **matrícula**.
3. Sistema valida a **matrícula**.
4. Sistema recupera os **empréstimos** do **aluno**.
5. Sistema apresenta os **empréstimos** existentes e solicita o **título** do **livro**.
6. Ator fornece o **título** do **livro**.
7. Sistema recupera dados do **livro**.
8. Sistema apresenta os dados do **livro** e solicita confirmação.
9. Ator confirma empréstimo.
10. Sistema registra **empréstimo** do **livro** com **data/hora do empréstimo** e **data de devolução**.
11. Sistema informa que empréstimo foi realizado com sucesso.

- A partir do caso de uso Emprestar Livro, apresentado anteriormente, podemos extrair o seguinte modelo de domínio:



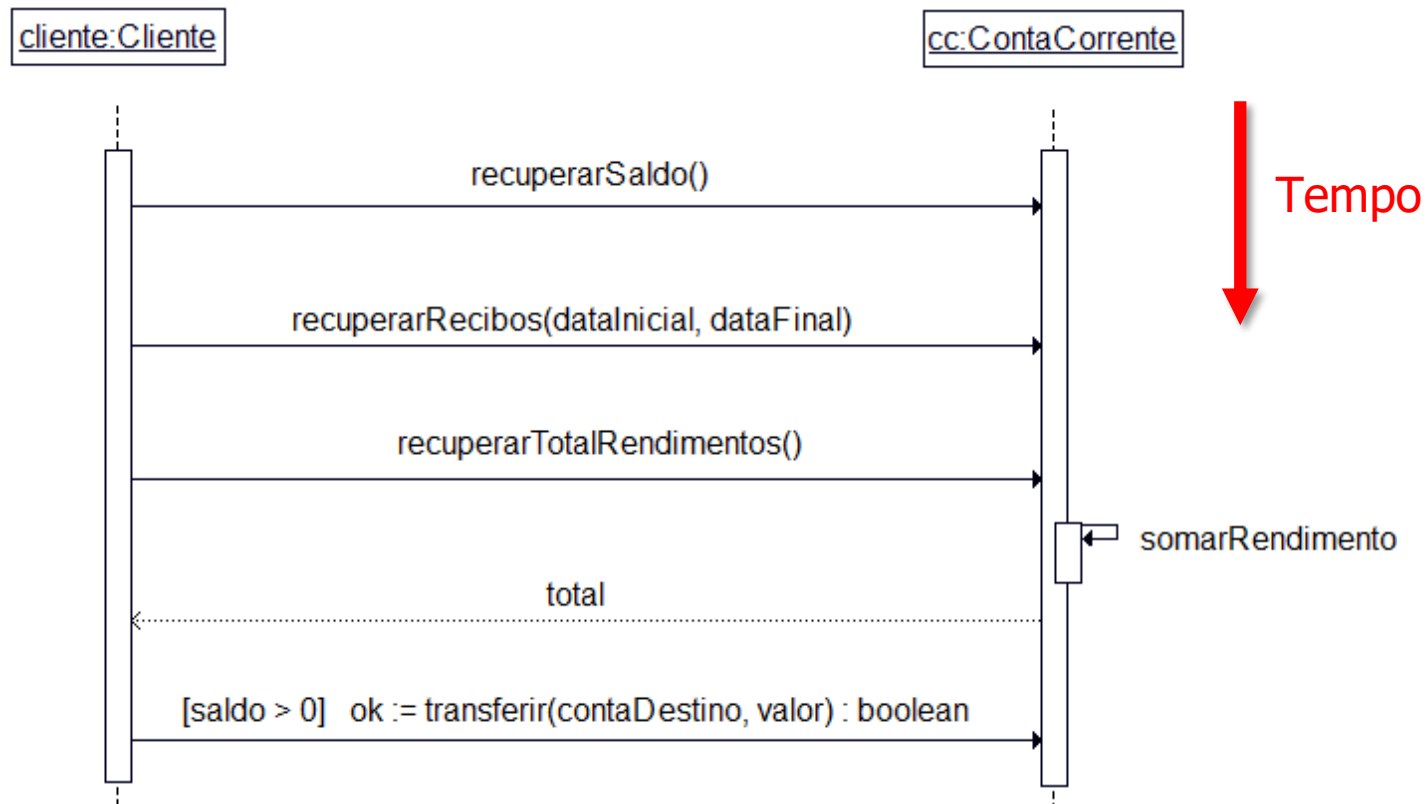
- Ou seja, no sistema de controle da biblioteca teremos objetos das classes **Aluno**, **Livro** e **Empréstimo**.
- Entretanto, esses objetos **não são suficientes**:
 - ✓ Que outros objetos são necessários?
 - ✓ Que comportamentos esses métodos devem ter?

- Por outro lado, sabemos que um sistema orientado a objetos funciona através da **troca de mensagens** entre objetos.

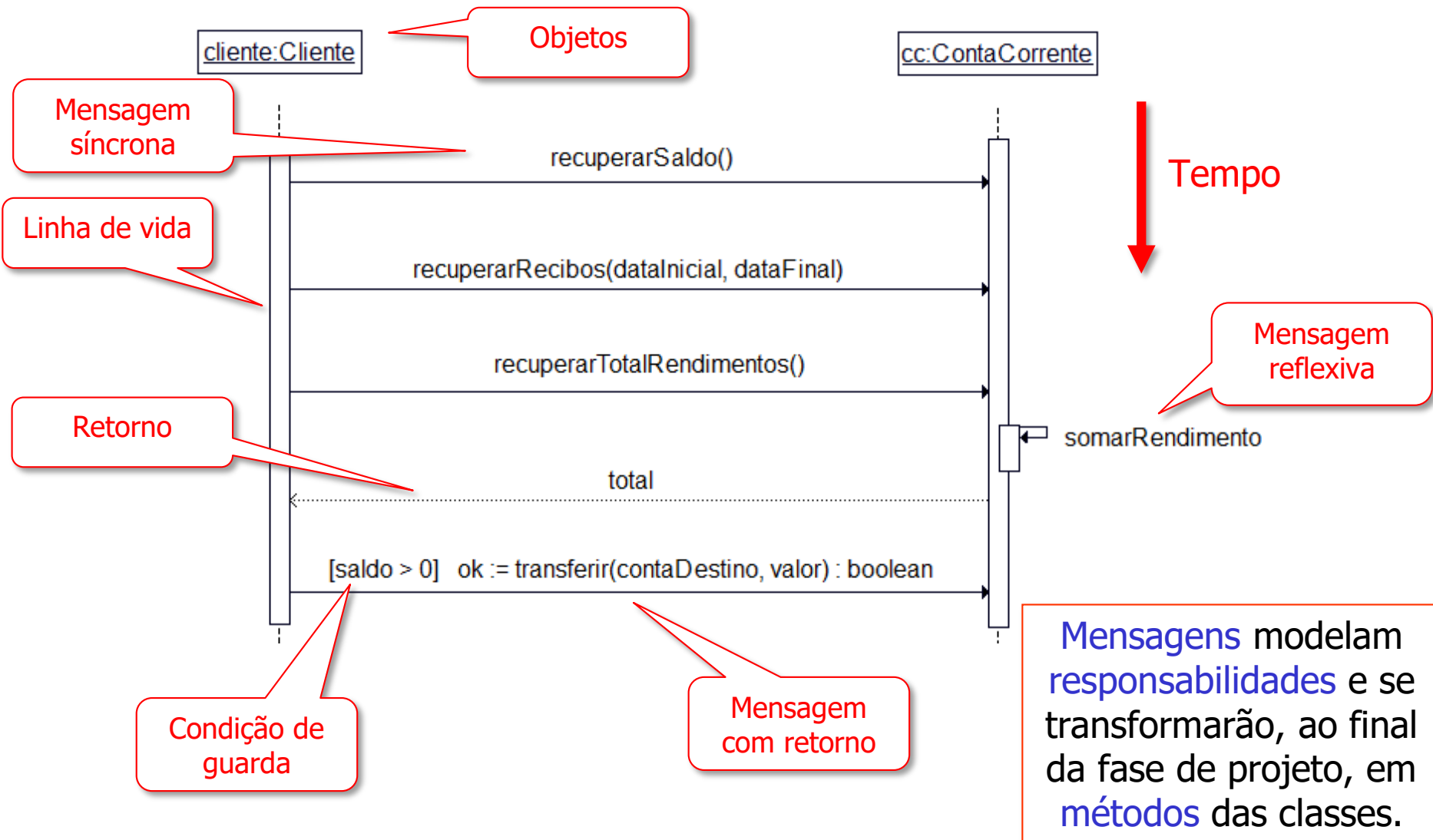


- É preciso, então, modelar essa troca de mensagens para descrevermos como o sistema irá funcionar.

- Os Diagramas de Sequência são usados para modelar as **interações** entre os objetos.
- Em outras palavras, esses diagramas modelam as **trocas de mensagens entre os objetos**, ou seja, ajudam a definir os objetos (classes) e seus comportamentos (métodos).
- Os Diagramas de Sequência podem ser usados nas fases de:
 1. Análise OO
 - ✓ São diagramas menos detalhados (mais abstratos) que descrevem os comportamentos do sistema.
 - ✓ São chamados de Diagramas de Sequência de Sistema (DSS).
 2. Projeto OO
 - ✓ São diagramas detalhados que descrevem como os objetos implementados no sistema interagem.
 - ✓ Representam de forma mais fiel como o sistema será implementado.



O Diagrama de Sequência modela a **troca de mensagens** entre **objetos** tendo por base uma **linha de tempo**, ou seja, a **ordem** em que as mensagens são enviadas é capturada nesse diagrama.



➤ Sintaxe geral das mensagens:

*[condição] valor := **mensagem**(parâmetro:tipo):tipo de retorno

onde:

mensagem	É a mensagem enviada para o outro objeto (único elemento obrigatório).
valor	É uma "variável" que vai receber o valor retornado pela mensagem.
parâmetro	É um valor enviado na mensagem (podem existir zero ou vários parâmetros).
tipo	É o tipo do parâmetro.
condição	É uma expressão booleana que define se a mensagem vai ou não ser enviada.
*	É um marcador que indica que a mensagem será enviada enquanto a condição for verdadeira (loop).

➤ Exemplos:

```
calcularSalario()
```

```
aumentarSalario(percentual)
```

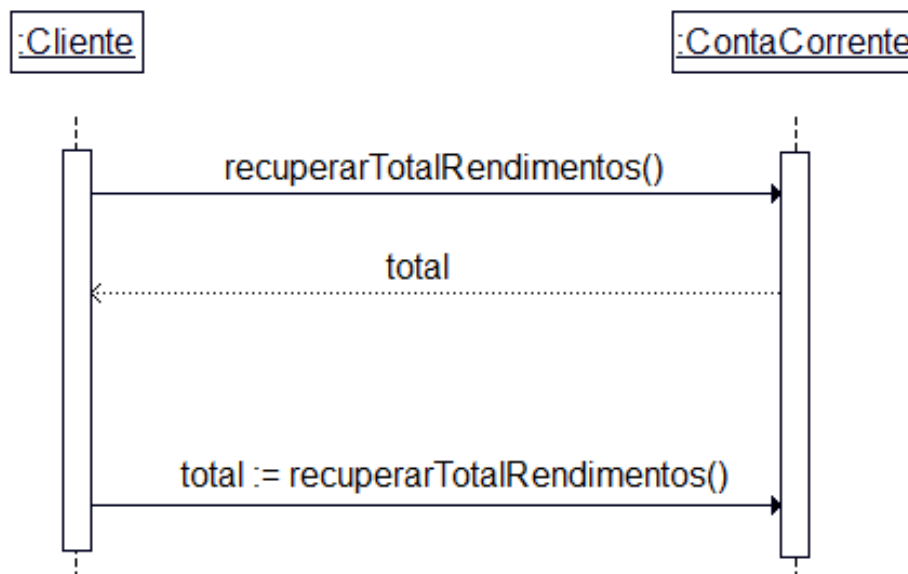
```
[percentual > 0] aumentarSalario(percentual)
```

```
novoSalario := aumentarSalario(percentual:float) : float
```

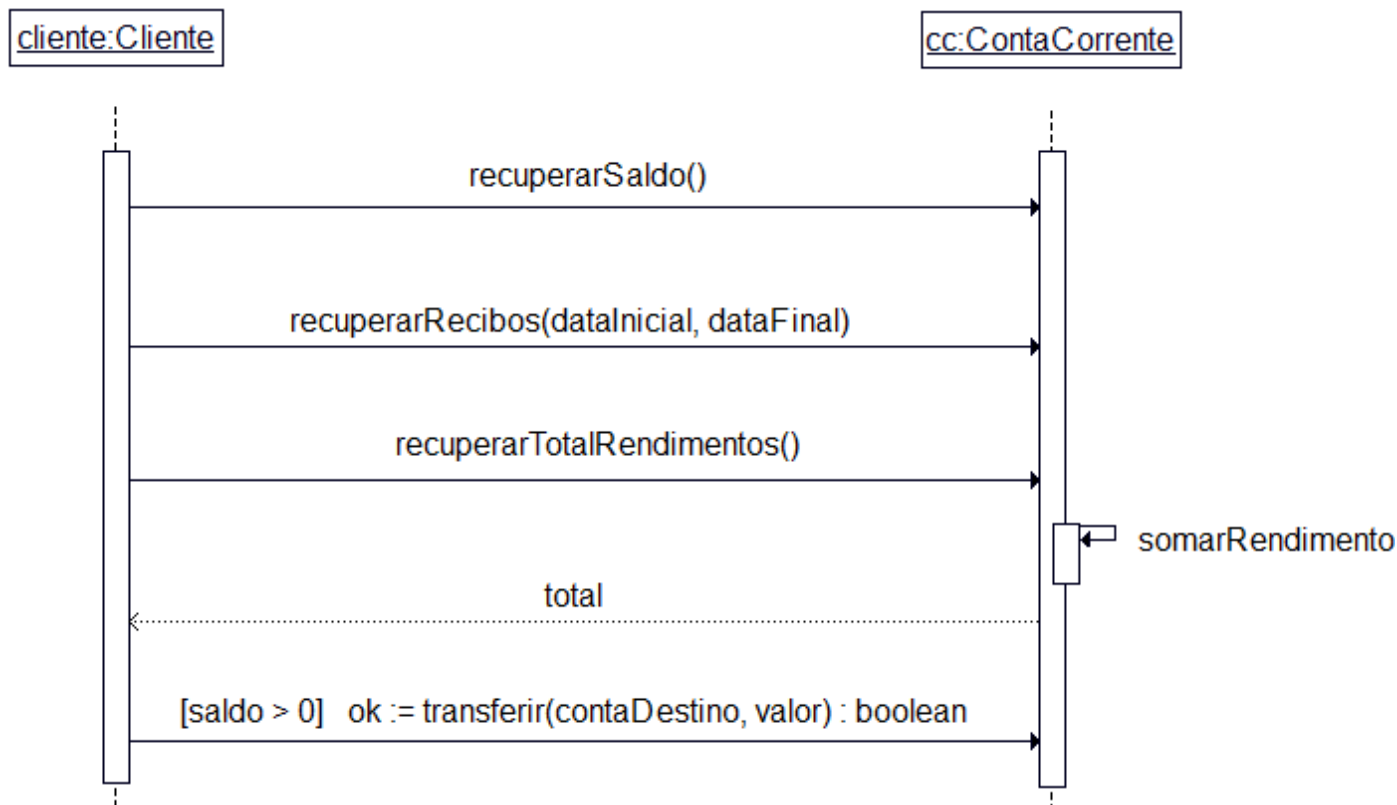
```
[percentual > 0] novoSalario := aumentarSalario(percentual)
```

```
*[existe item] registrarItem(código, quantidade)
```

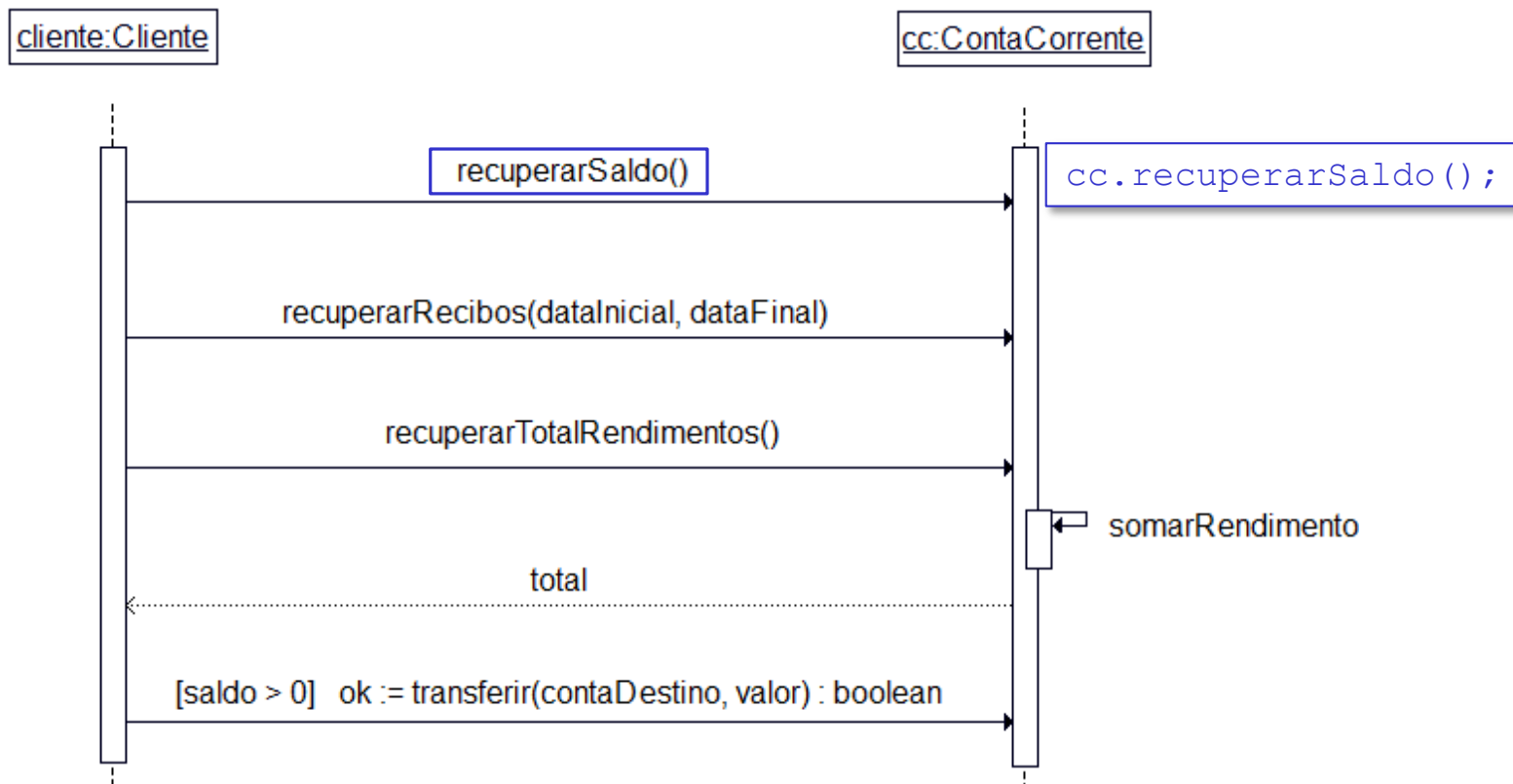
- Existem duas formas de modelar o **valor retornado** pela mensagem:
- a) Retorno (seta pontilhada)
 - b) Atribuição ($:=$)



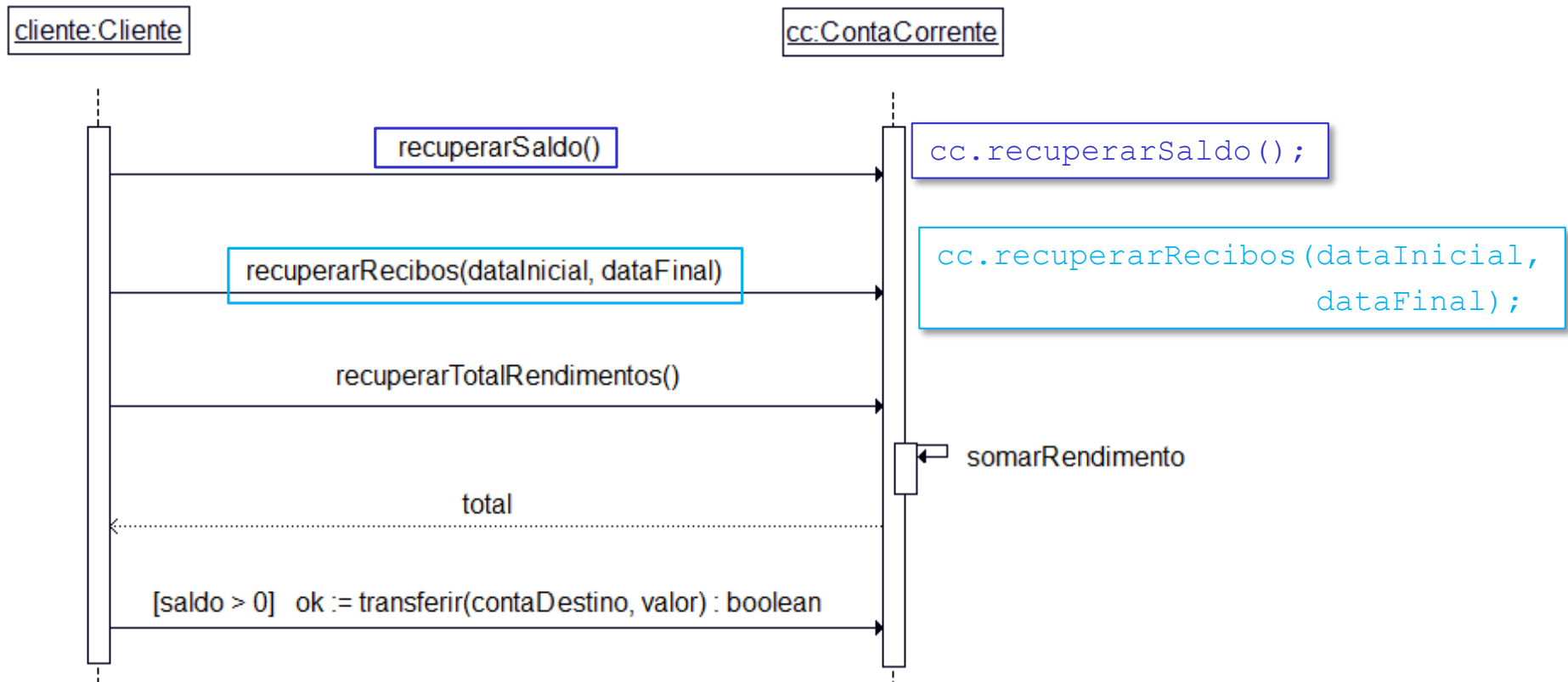
- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



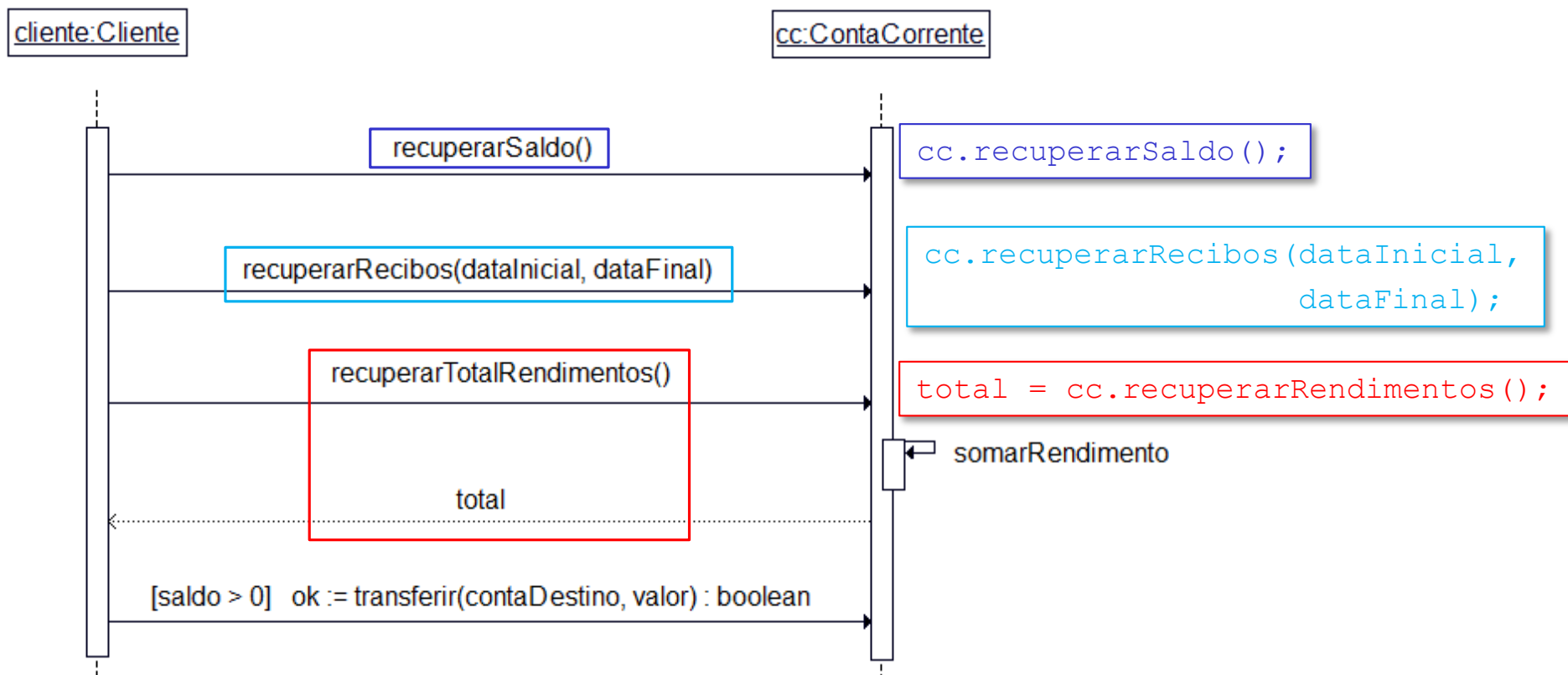
- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



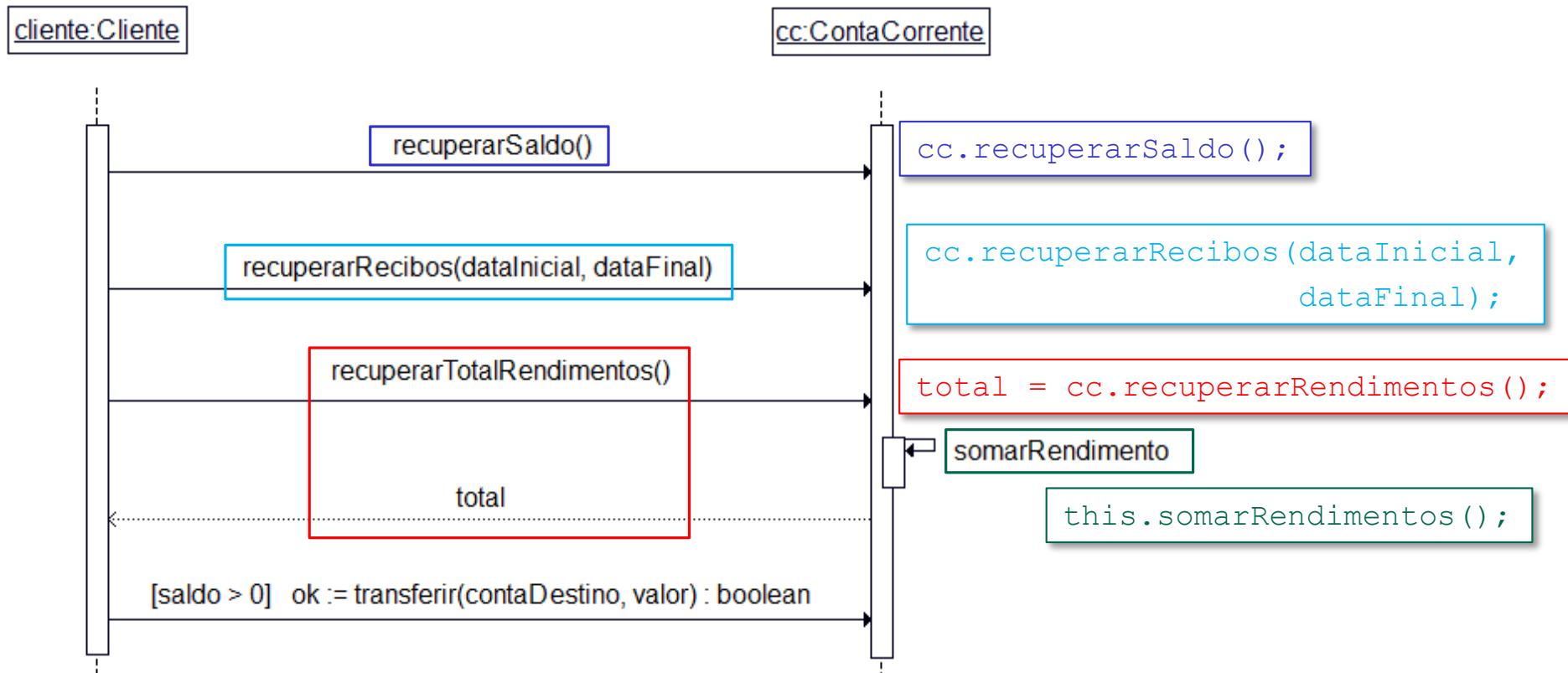
- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



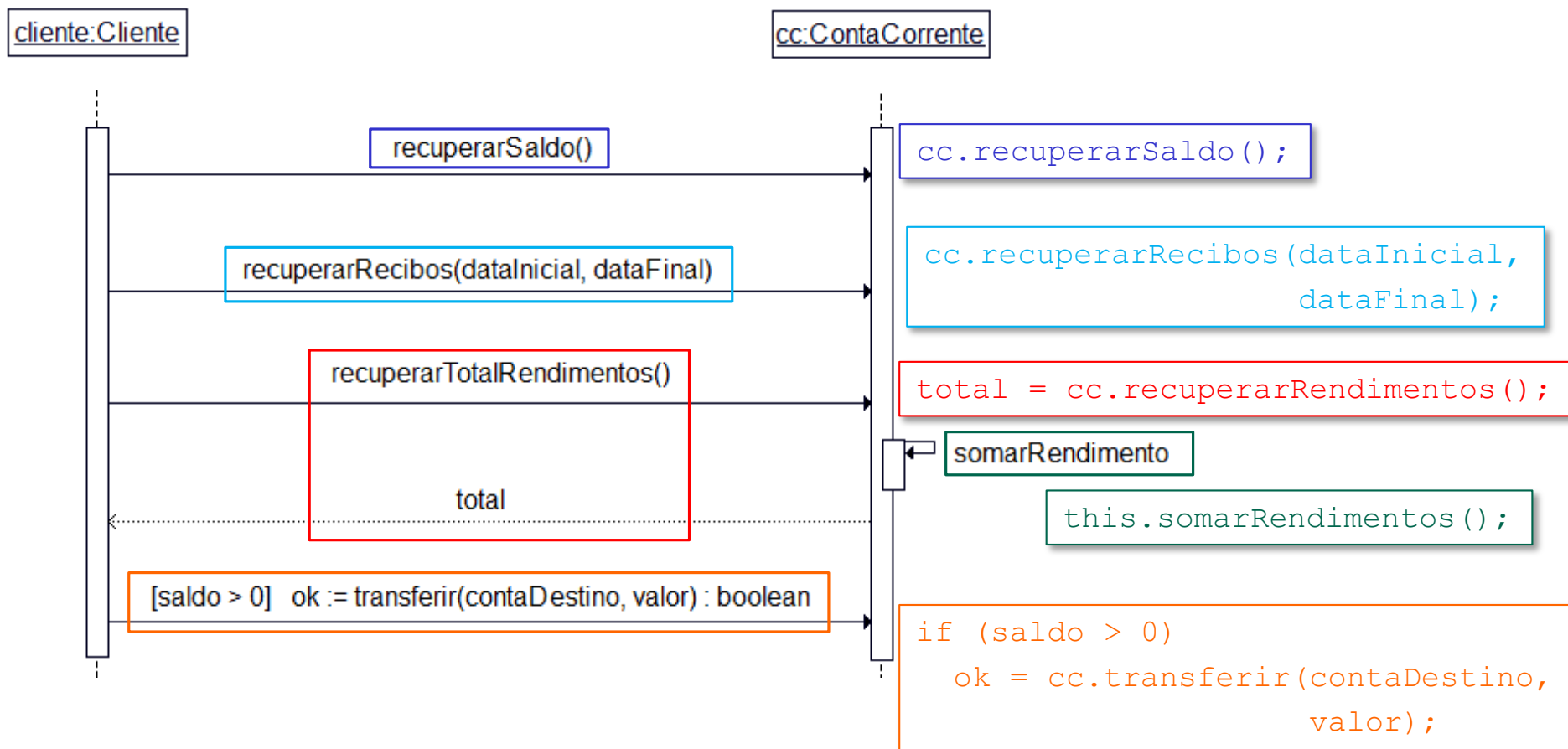
- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



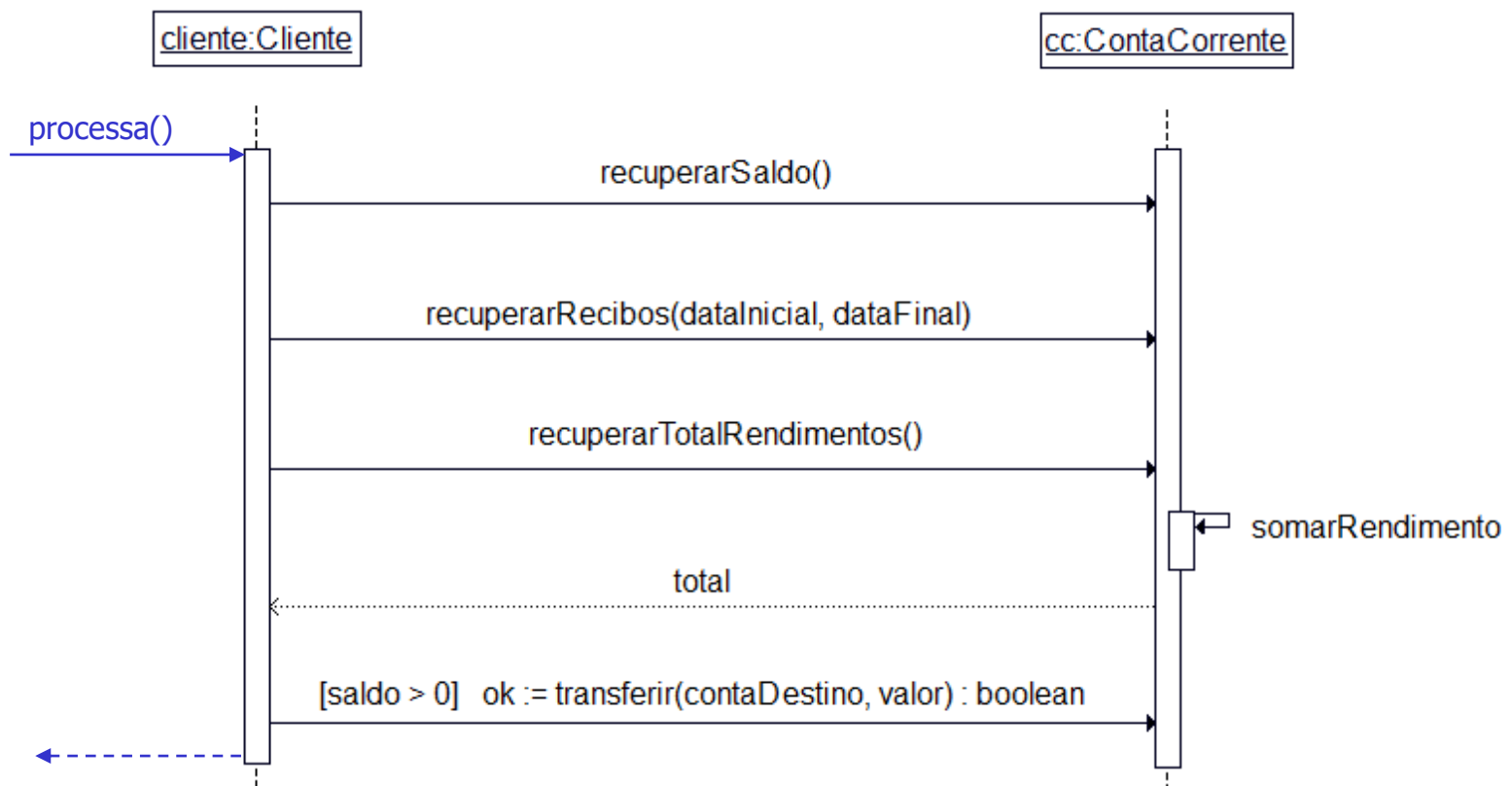
- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



- Traduzindo essas mensagens para chamadas de **métodos** na linguagem Java:



- Para entender o cenário completo, imagine que o **cliente** recebeu a mensagem síncrona **processa()**:



- Nesse caso, as classes **Cliente** e **ContaCorrente** teriam os seguintes métodos:

```
public class Cliente
{
    public void processa()
    {
        cc.recuperarSaldo();

        cc.recuperarRecibos(dataInicial,
                           dataFinal);

        total = cc.recuperarRendimentos();

        if (saldo > 0)
        {
            ok = cc.transfere(contaDestino,
                             valor);
        }
    }
}
```

```
public class ContaCorrente
{
    public float recuperarSaldo() {
    }

    public void recuperarRecibos(Date dataIni,
                                Date dataFim) {
    }

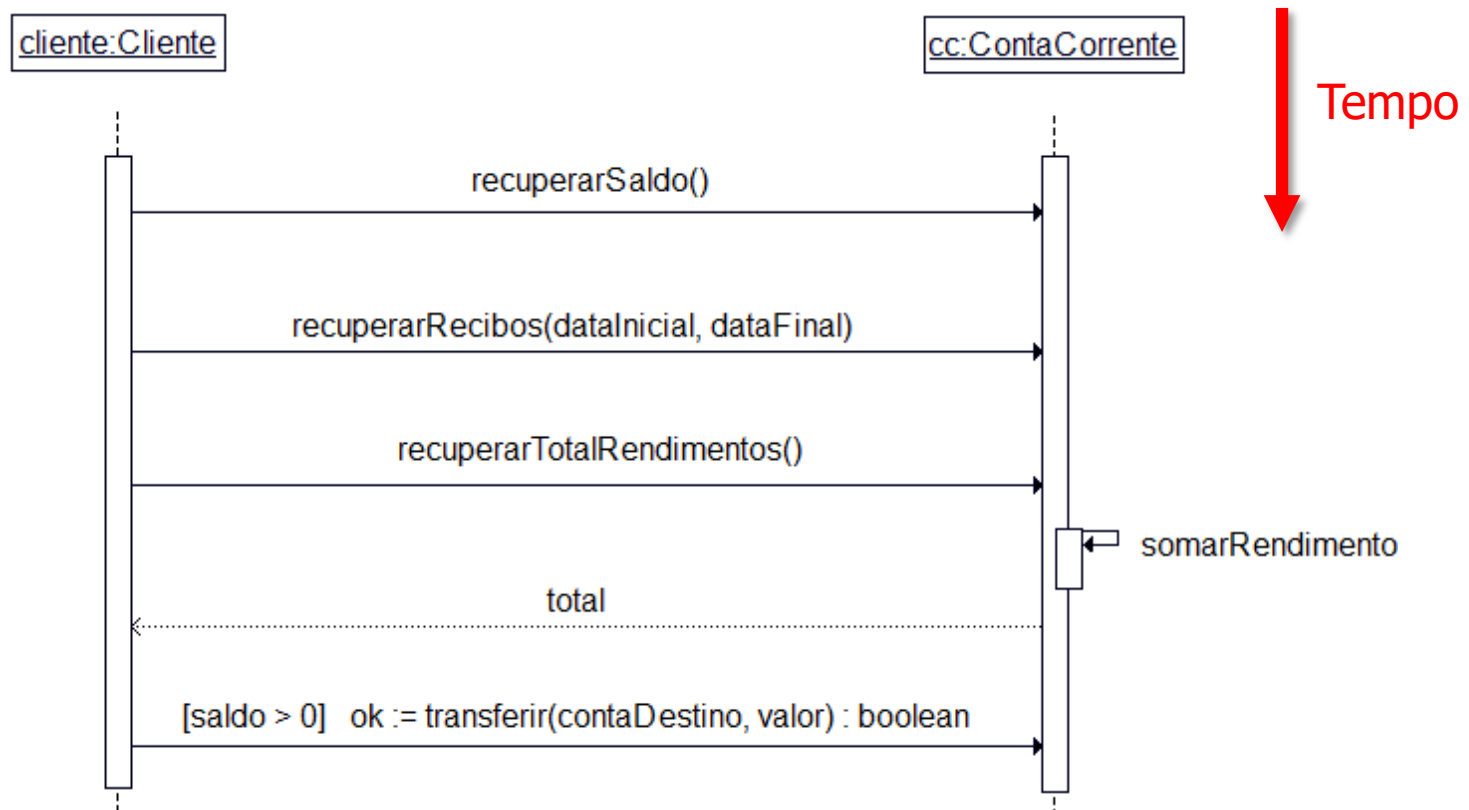
    public float recuperarRendimentos() {
        total = this.somarRendimentos();
        return total;
    }

    public void somarRendimentos() {
    }

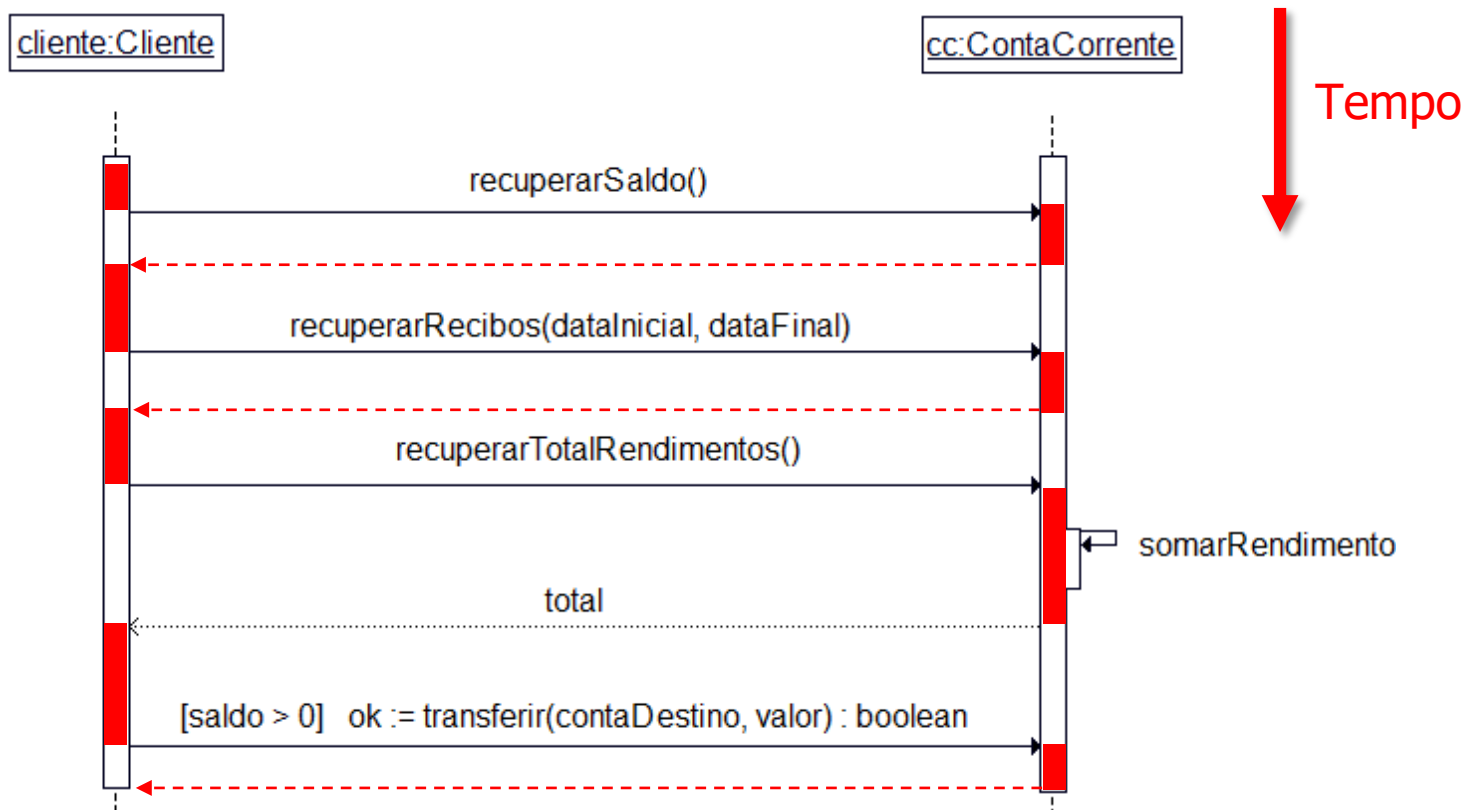
    public boolean transfere(ContaCorrente destino,
                             float valor) {
    }
}
```

1. Mensagem **síncrona** (seta preenchida) é aquela em que o objeto chamador envia a mensagem para outro objeto e **fica aguardando o retorno** do objeto chamado antes de prosseguir.
 - ✓ Processamento *mono thread*: apenas **um** objeto está em execução em um dado momento.
2. Mensagem **assíncrona** (seta sem preenchimento) é aquela em que o objeto chamador envia a mensagem para outro objeto e **continua o processamento sem aguardar o retorno** do objeto chamado.
 - ✓ Processamento *multi thread*: **vários** objetos podem estar em execução em um dado momento.

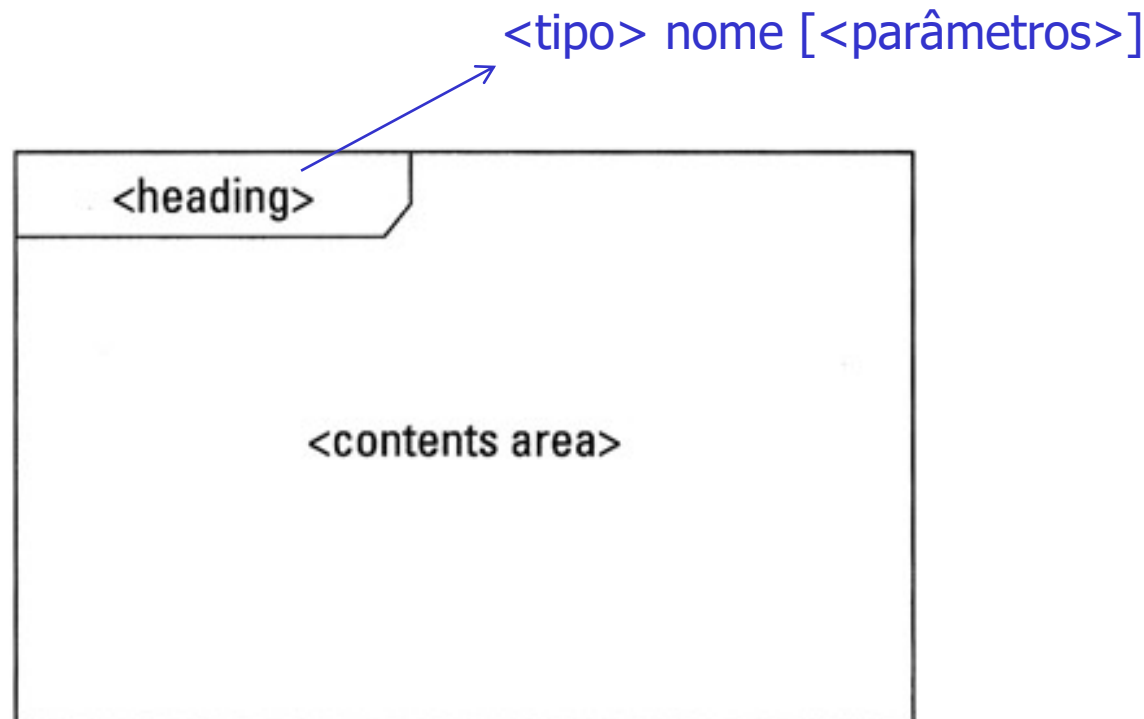
- Como as mensagens são síncronas, apenas **um método** de um objeto está em execução em determinado momento:



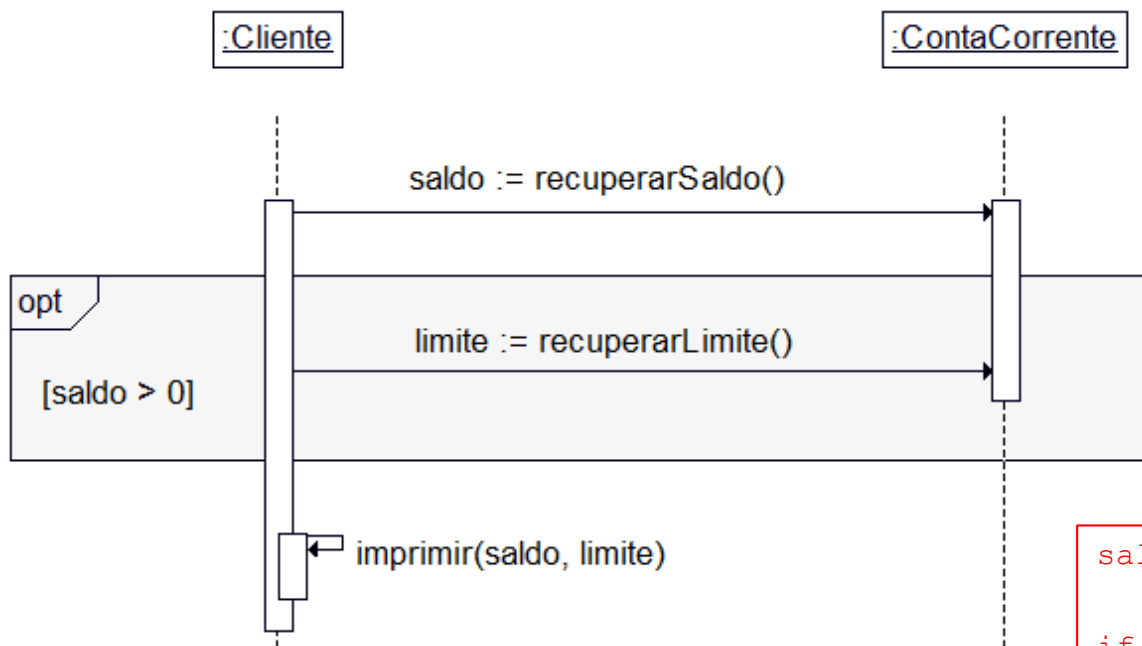
- Como as mensagens são síncronas, apenas **um método** de um objeto está em execução em determinado momento:



- Uma alteração importante na UML 2.0 foi a introdução de **frames** e seus **tipos**.
- Frame: define um **contexto parametrizável** em um diagrama de seqüência, possibilitando desvios de fluxo e também o seu **reuso**.



- **optional (opt)**: define um bloco de mensagens que são enviadas somente se uma determinada condição for satisfeita (condição de guarda). Equivalente a um *if-then*.

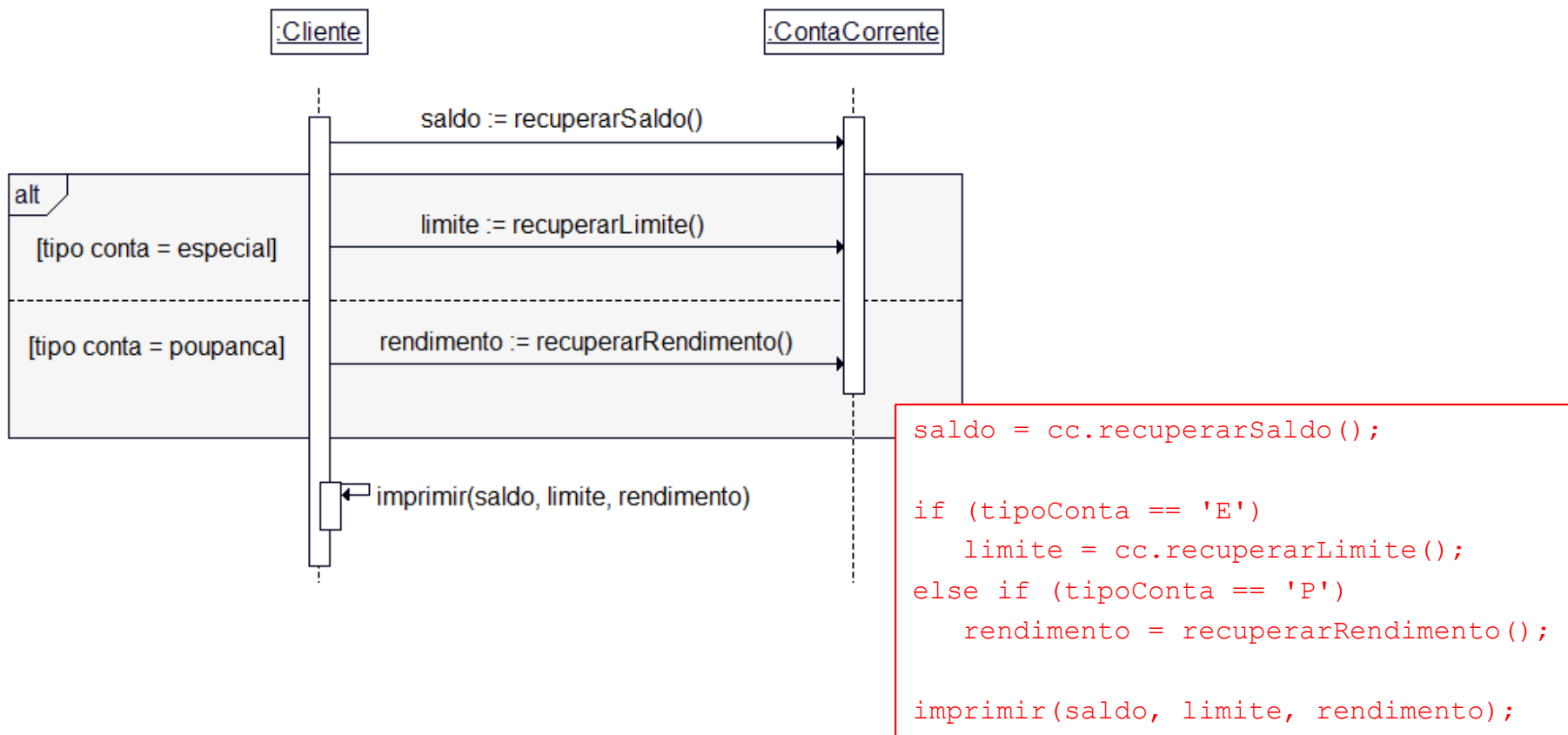


```
saldo = cc.recuperarSaldo();

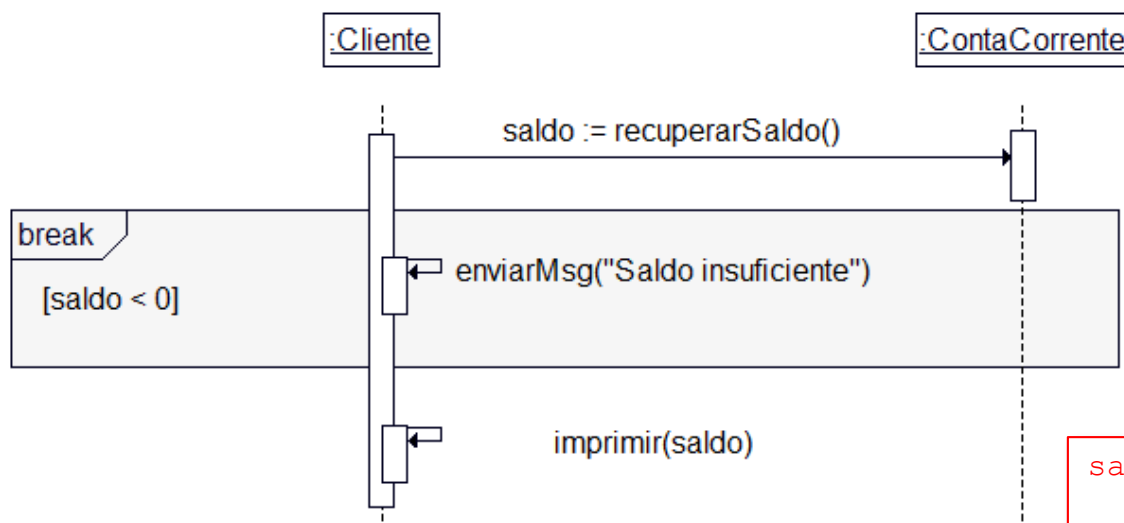
if (saldo > 0)
    limite = cc.recuperarLimite();
end

imprimir(saldo, limite);
```

- **alternative (alt):** define blocos de mensagens que são enviadas se as condições associadas a cada um deles for satisfeita. Apenas um conjunto é executado. Equivalente a um *if-then-elseif-then-elseif-then-elseif...*



- **break**: equivalente ao **opt**, só que após a execução do bloco a execução de todo o diagrama é **interrompida** (*return*).

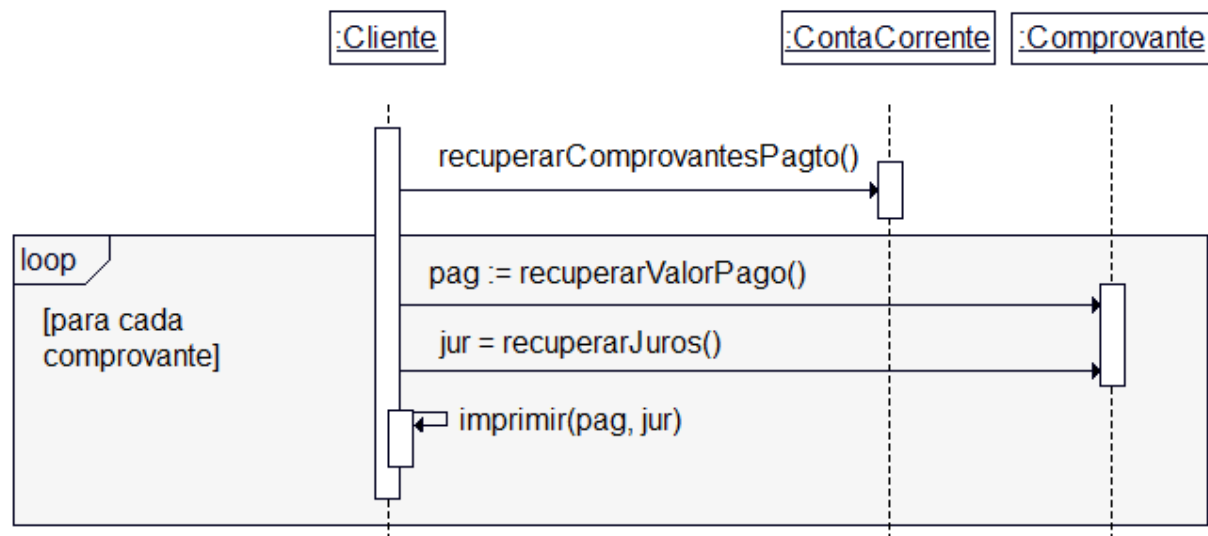


```
saldo = cc.recuperarSaldo();

if (saldo < 0) {
    enviarMsg("Saldo insuficiente");
    return;
}

imprimir(saldo);
```

- **loop**: define um bloco de mensagens que são enviadas em *loop*.



```
comprovantes = cc.recuperarComprovantesPagto();

for (Comprovante c in comprovantes) {
    pag = c.recuperarValorPago();
    jur = c.recuperarJuros();
    imprimir(pag, jur);
}
```

- Os Diagramas de Sequência de Sistema (DSS) foram propostos por Craig Larman em 1998.
- Um DSS é um retrato que mostra os eventos que os atores geram, sua ordem e os eventos e operações do sistema.
- No DSS o sistema é apresentado como uma caixa-preta, ou seja, não são modelados os objetos que fazem parte do sistema (estrutura interna do sistema).

- Baseados em quê podemos modelar um DSS?
 - ✓ Nos **Casos de Uso**, pois os eventos e comportamentos esperados do sistema estão lá!

- Como?
 - ✓ Criando **um Diagrama de Sequência para cada caso de uso**, ou seja, vamos modelar o comportamento do sistema **inspecionando** cada caso de uso.
 - ✓ O sistema será representado como uma **caixa-preta**, ou seja, **não** serão modelados os objetos que compõem a estrutura interna do sistema.
 - ✓ O foco deve estar nos **eventos gerados pelos atores**, como o sistema **processa esses eventos** (operações do sistema) e as **respostas** geradas pelo sistema.

- Os casos de uso descrevem como os atores interagem com o sistema de software que estamos interessados em criar.
- Durante essa interação, um ator gera **eventos** para o sistema, geralmente solicitando alguma **operação** de sistema para tratar o evento.
 - ✓ Por exemplo, quando um caixa de supermercado entra com o código de um item, podemos dizer que ele está solicitando ao sistema PDV que registre a venda daquele item.
- Assim, um **evento inicia uma operação** no sistema.
- O texto do caso de uso sugere o evento e o DSS o torna concreto e explícito.

➤ Exemplo 1:

Nome: Empréstimo de livro

Objetivo: Realizar o empréstimo de livro ao aluno

Ator: Bibliotecária

Trigger: Ator acessa a página de **empréstimo** de **livros**

Fluxo Principal:

1. Sistema solicita a **matrícula** do **aluno**.
2. Ator informa a **matrícula**.
3. Sistema valida a **matrícula**.
4. Sistema recupera os **empréstimos** do **aluno**.
5. Sistema apresenta os **empréstimos** existentes e solicita o **título** do **livro**.
6. Ator fornece o **título** do **livro**.
7. Sistema recupera dados do **livro**.
8. Sistema apresenta os dados do **livro** e solicita confirmação.
9. Ator confirma empréstimo.
10. Sistema registra **empréstimo** do **livro** com **data/hora do empréstimo** e **data de devolução**.
11. Sistema informa que **empréstimo** foi realizado com sucesso.

UVA ● Diagrama de Sequência de Sistema

⋈
:Bibliotecaria

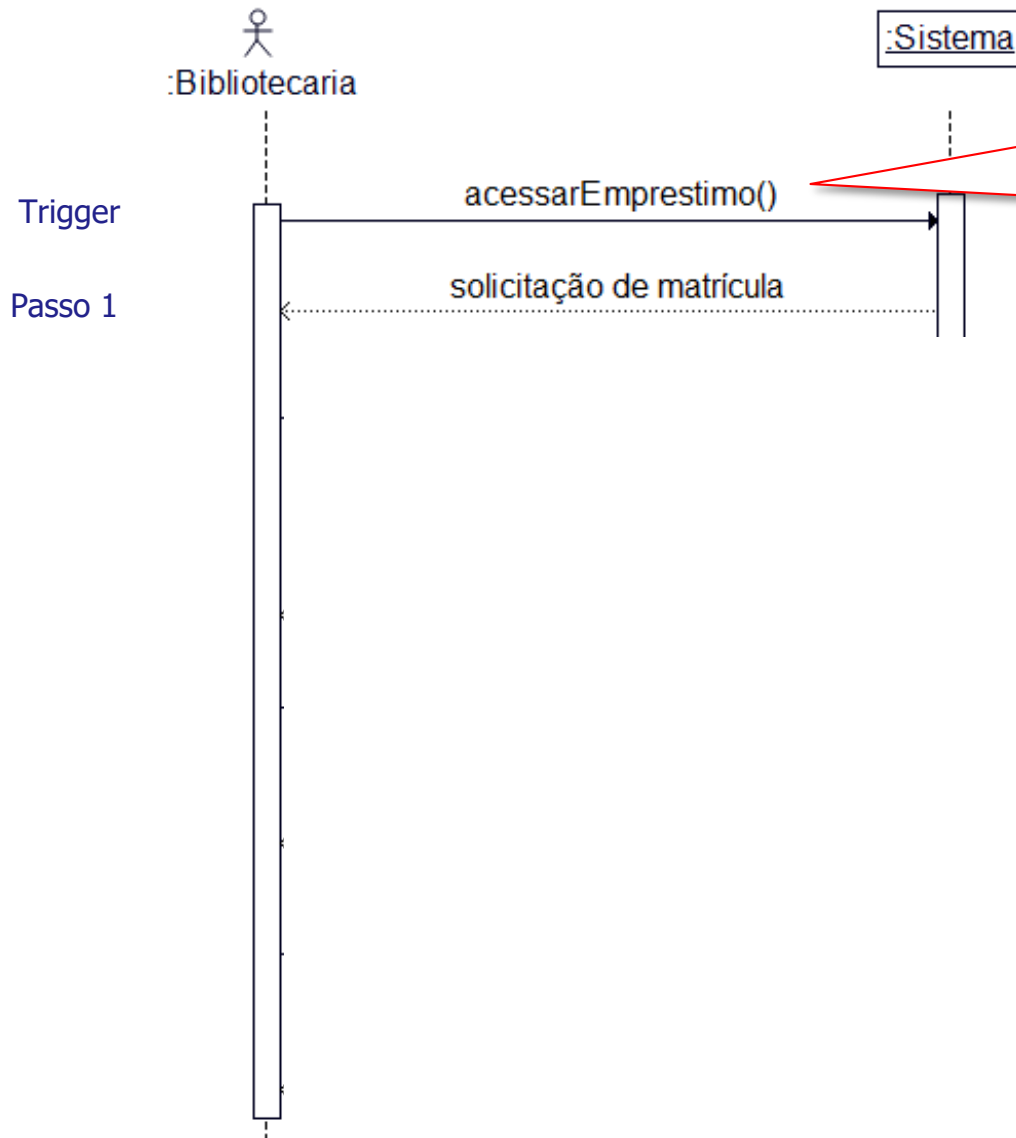


:Sistema



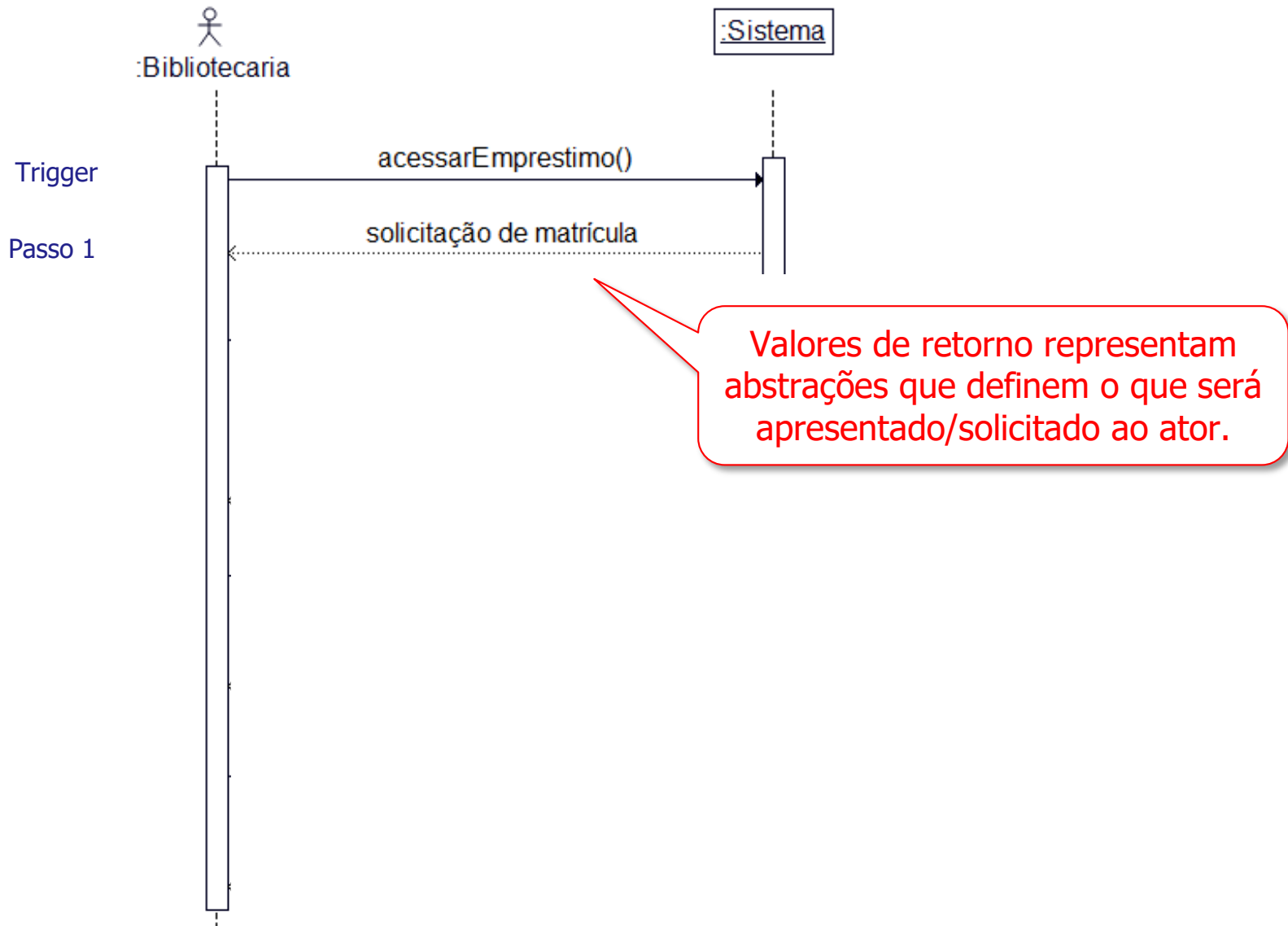
No DSS modelamos apenas os atores e o sistema é representado como um único objeto (caixa-preta).

UVA● Diagrama de Sequência de Sistema

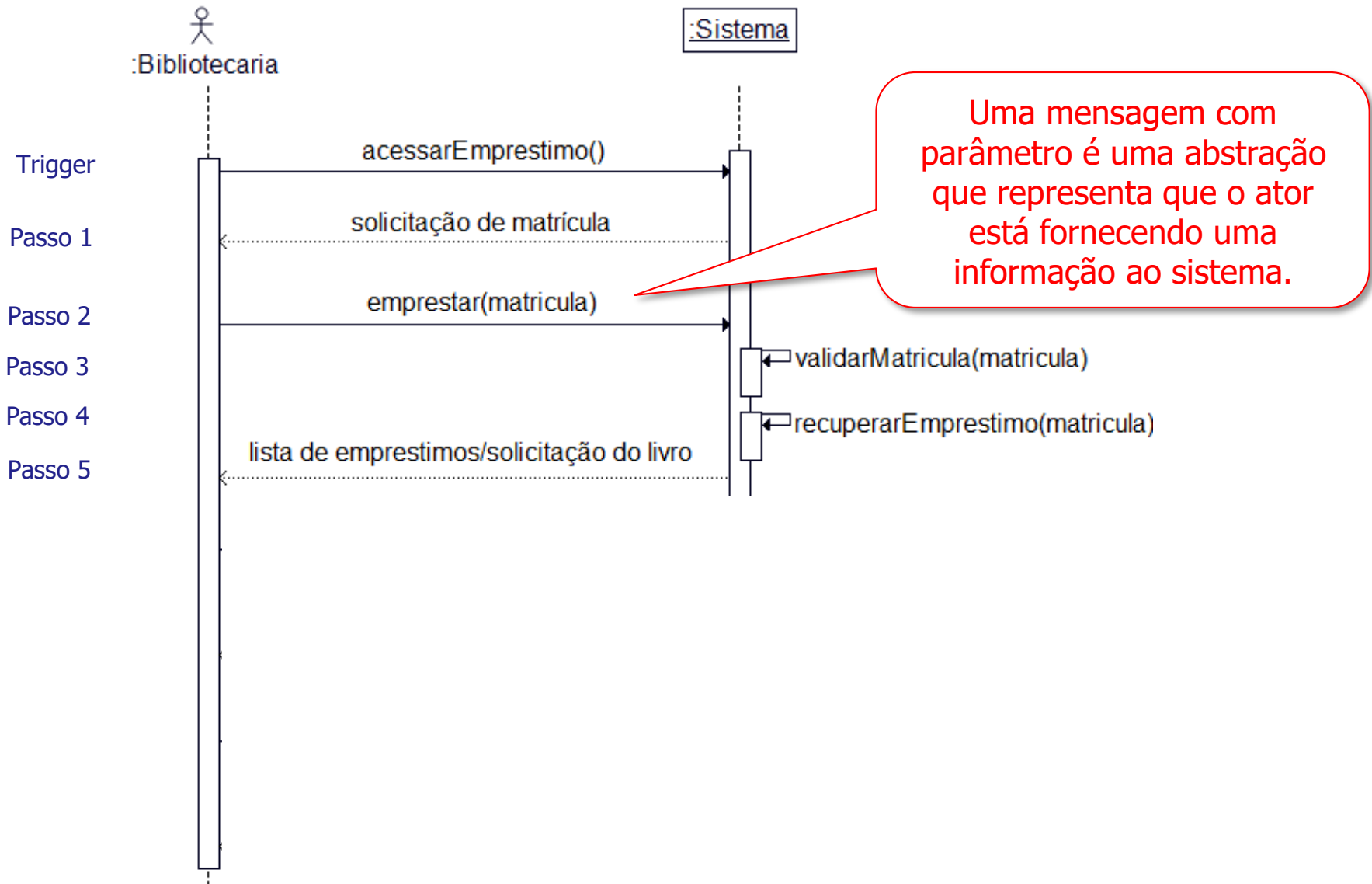


Eventos representam solicitações que os atores fazem ao sistema e que disparam operações para tratar esses eventos.

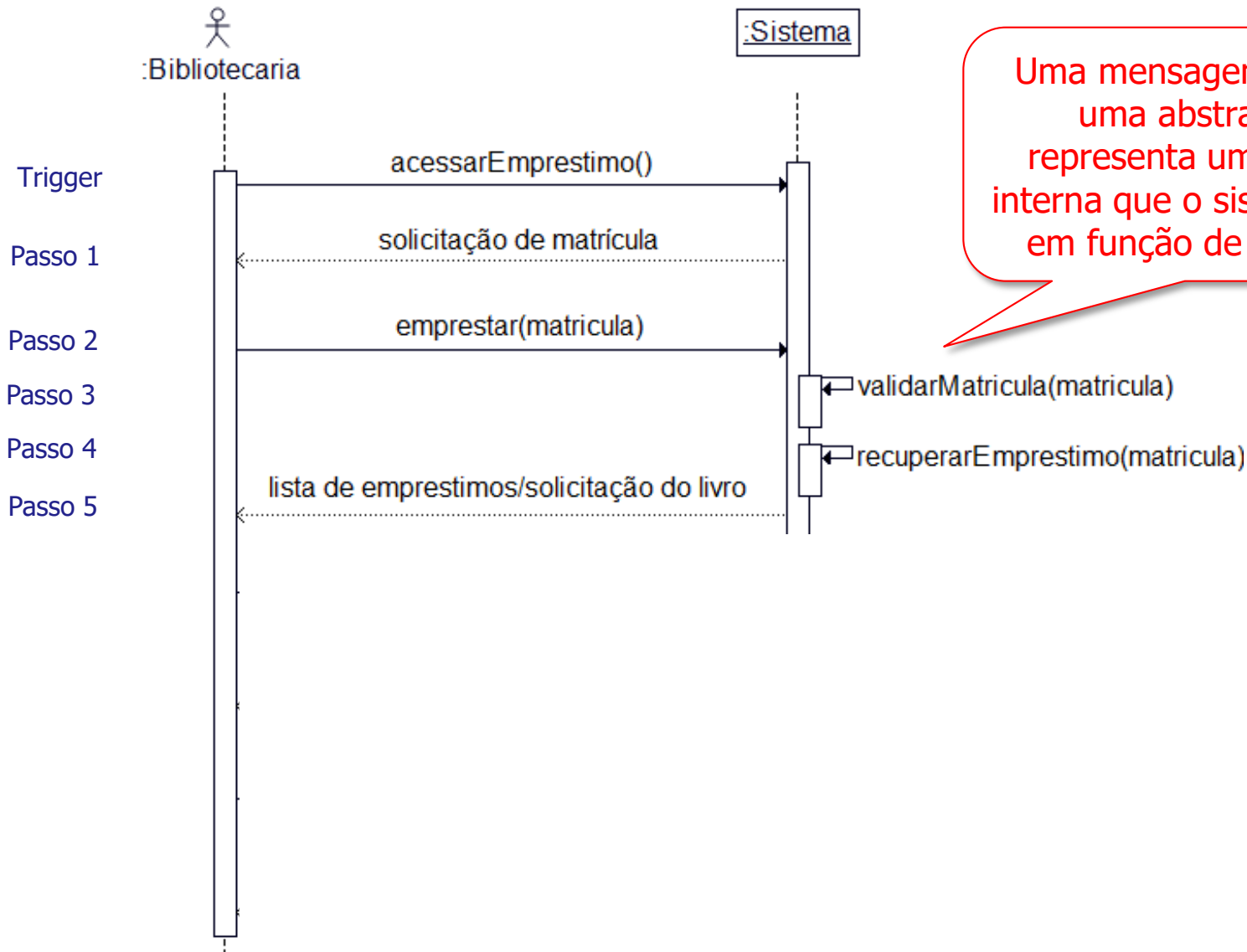
UVA● Diagrama de Sequência de Sistema



UVA● Diagrama de Sequência de Sistema

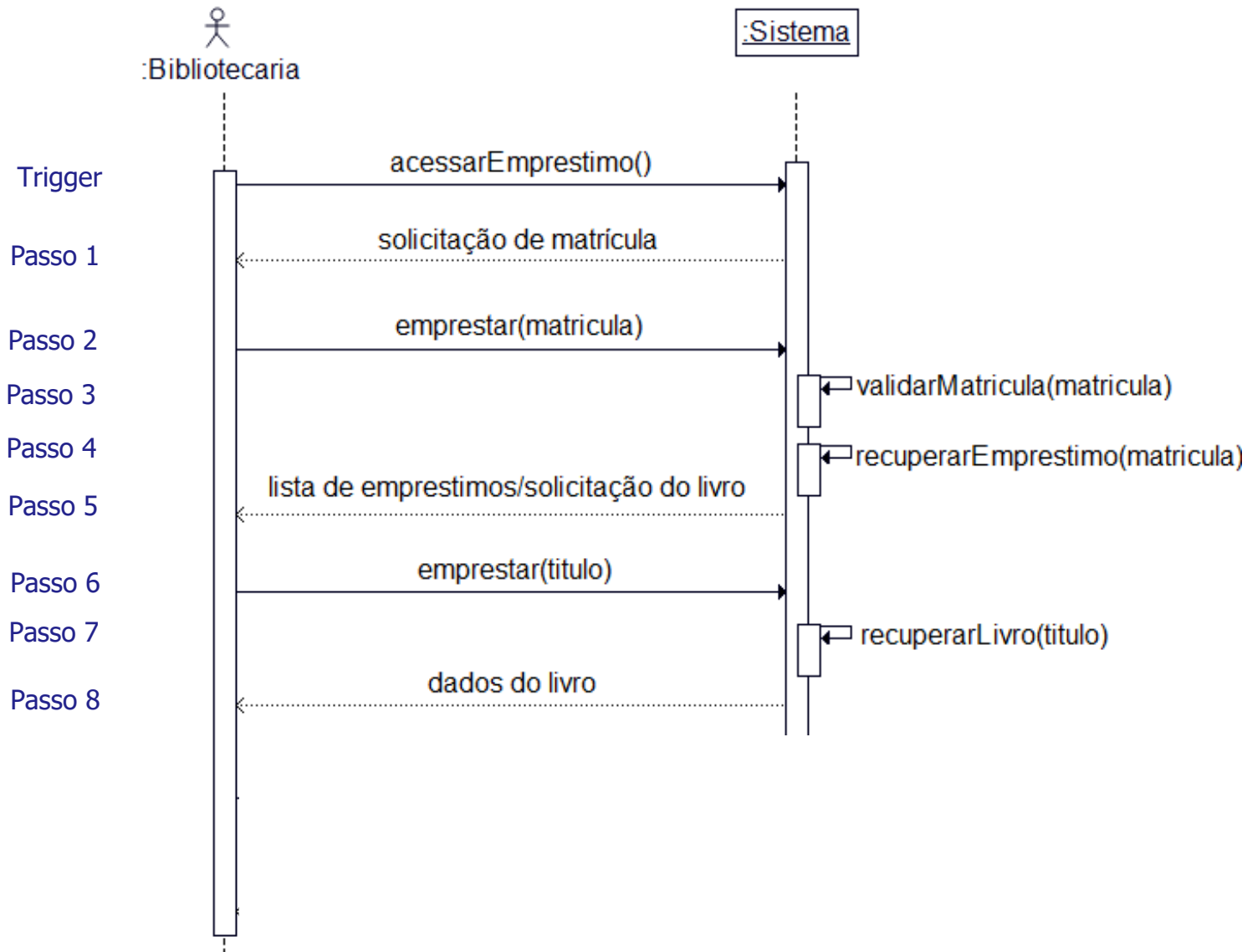


UVA Diagrama de Sequência de Sistema

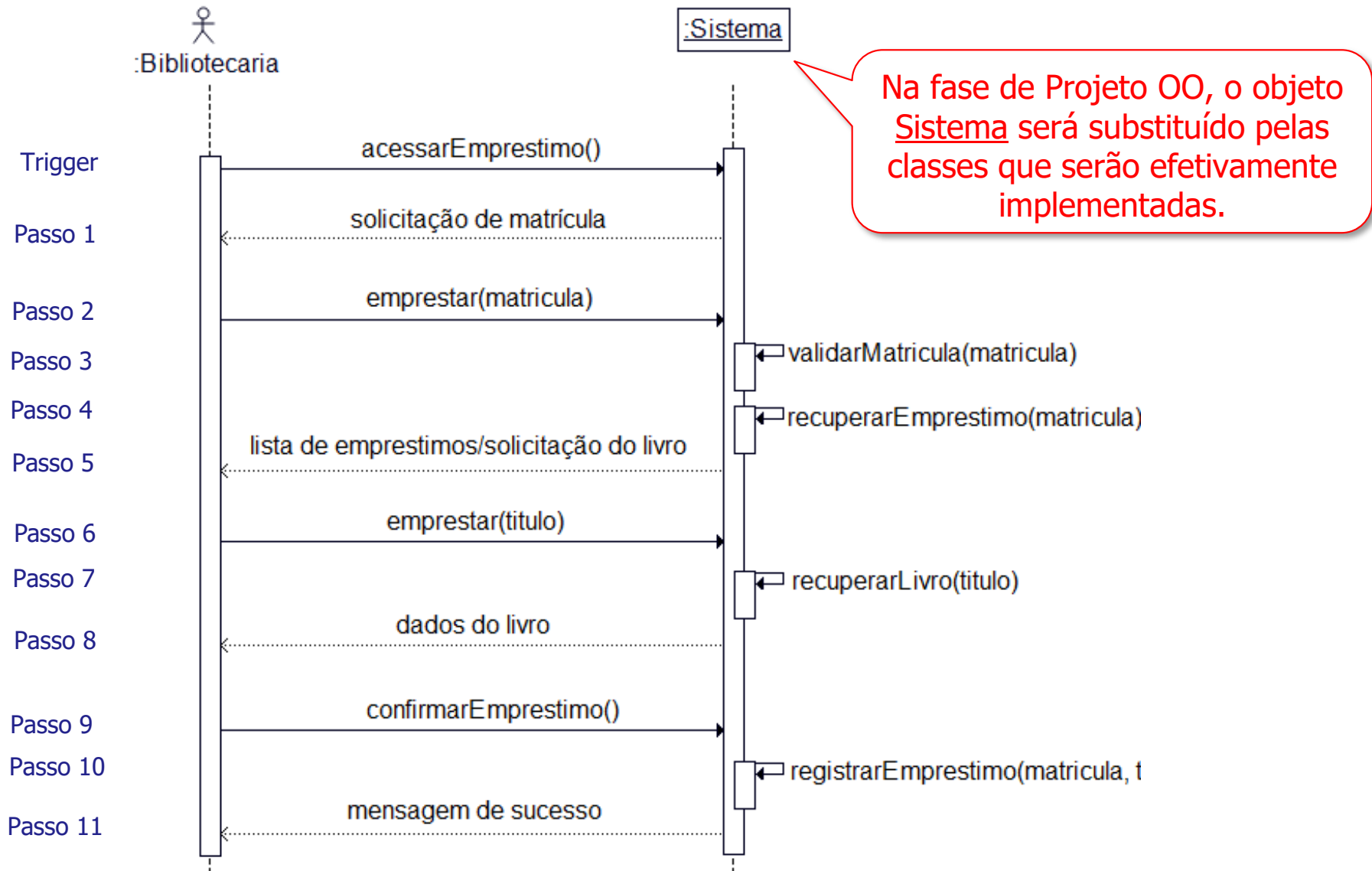


Uma mensagem reflexiva é uma abstração que representa uma operação interna que o sistema executa em função de um evento.

UVA Diagrama de Sequência de Sistema



UVA ● Diagrama de Sequência de Sistema



- A partir do DSS é possível identificar os comportamentos (responsabilidades) do sistema:

Sistema

acessarEmprestimo()
emprestar(matrícula)
validarMatricula(matrícula)
recuperarEmprestimo(matrícula)
emprestar(título)
recuperarLivro(título)
confirmarEmprestimo()
registrarEmprestimo(matrícula, título)

➤ Exemplo 2:

Nome: Realizar venda

Objetivo: Realizar a venda de produtos em um supermercado (autoatendimento)

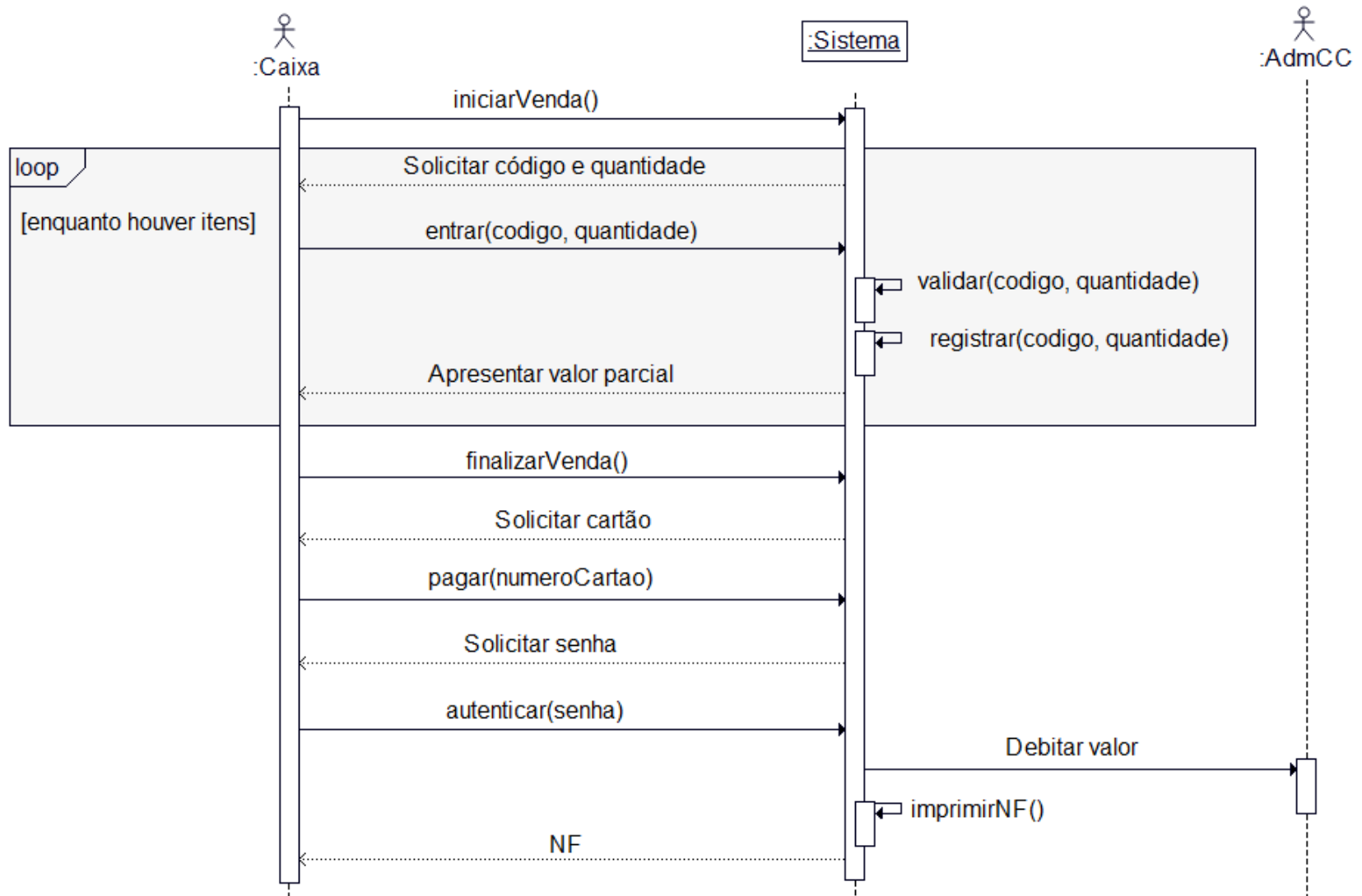
Ator: Caixa

Trigger: Ator inicia a venda.

Fluxo Principal:

1. Sistema solicita o código do produto e a quantidade.
 2. Ator informa o código do produto e a quantidade.
 3. Sistema valida o código do produto e a quantidade.
 4. Sistema registra o produto e a quantidade.
 5. Sistema apresenta o total parcial da venda.
 6. Ator seleciona o fim da venda.
 7. Sistema apresenta o valor total e solicita a inserção do cartão de crédito.
 8. Ator insere o cartão de crédito.
 9. Sistema solicita a senha.
 10. Ator informa a senha.
 11. Sistema debita o valor no cartão de crédito.
 12. Sistema imprime a NF.
- Os passos de 1 a 5 serão repetidos até terminarem os produtos

UVA Diagrama de Sequência de Sistema



- A partir do DSS é possível identificar os comportamentos (responsabilidades) do sistema:

Sistema

iniciarVenda()
entrar(código, quantidade)
validar(código, quantidade)
registrar(código, quantidade)
finalizarVenda()
pagar(numeroCartão)
autenticar(senha)
imprimirNF()

➤ Por que modelar um DSS?

- ✓ Uma questão interessante e útil no projeto de software é a seguinte: que eventos estão sendo enviado para o sistema?
- ✓ Por quê? Porque temos que projetar o software para tratar esses eventos e responder a eles.
- ✓ Basicamente, um sistema de software reage a três coisas:
 - ✓ Eventos externos de atores (seres humanos, dispositivos ou outros sistemas).
 - ✓ Eventos de tempo.
 - ✓ Falhas ou exceções (frequentemente gerados por fontes externas).
- ✓ Portanto, é útil saber precisamente quais são os eventos externos de entrada e as operações do sistema, pois eles são essenciais na análise do comportamento do sistema.